

CHAPTER 6: PATTERN MINING (KHAI PHÁ MẪU)

Mục lục

6.1. What Are Patterns?	2
6.2. Pattern Discovery (Khám phá Mẫu)	2
6.3. Frequent Itemsets (Tập phổ biến)	4
6.3.1. Transactional Database (TDB) và các khái niệm liên quan	4
6.3.2. Frequent Itemsets (Patterns)	5
6.4. Association Rules (Luật kết hợp)	6
6.4.1. From Frequent Itemsets to Association Rules	6
6.4.2. Hai thước đo để đánh giá sức mạnh của một luật kết hợp	7
6.4.3. Mining Frequent Itemsets and Association Rules	7
6.4.4. There Are Too Many Frequent Patterns!	8
6.4.4.1. Closed Patterns (Mẫu đóng)	9
6.4.4.2. Max-Patterns (Mẫu cực đại)	9
6.5. Bài toán khám phá Luật kết hợp	10
6.5.1. Nguyên lý Apriori (Apriori Principle)/ The Downward Closure Property	10
6.5.1.1. Các bước của thuật toán Apriori (A Candidate Generation & Test Approach)	11
6.5.1.2. Example	12
6.5.1.3. Improvements and Alternatives	12
6.5.2. Thuật toán FP-Growth	14
6.5.2.1. Điều gì xảy ra nếu FP-tree không vừa trong bộ nhớ?	20
6.5.2.2. CLOSET+: Mining Closed Itemsets by Pattern-Growth	21
6.5.2.3. Mở rộng FP-Growth	21
6.6. Độ đo tính lý thú của luật kết hợp - Đánh giá Mẫu	22
6.6.1. Lift (Độ nâng)	22
6.6.2. χ^2 (Chi-squared)	23
6.6.3. Lift and χ^2 : Are They Always Good Measures? Và bàn về các độ đo bất biến với rỗng	24
6.6.4. Imbalance Ratio with Kulczynski Measure	25
6.6.5. What Measures to Choose for Effective Pattern Evaluation?	26
6.6.6. Một số Độ đo tính lý thú khác	26

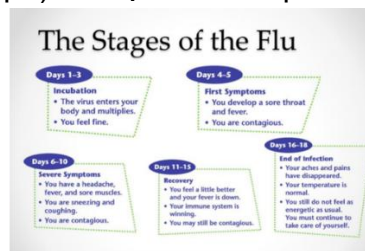
Pattern Mining (Khai phá Mẫu) là một nhánh của Khai phá Dữ liệu, tập trung vào việc tìm kiếm các quy luật, sự lặp lại, hoặc các cấu trúc thú vị và tiềm ẩn trong các tập dữ liệu lớn.

6.1. What Are Patterns?

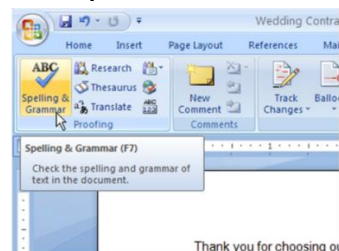
- **Mẫu (Pattern):** Là một tập hợp các item (mặt hàng), chuỗi con (subsequence), hoặc cấu trúc con (substructure) **thường xuyên xuất hiện cùng nhau (hoặc có tương quan mạnh)** trong một tập dữ liệu.
- Các mẫu này đại diện cho các **thuộc tính nội tại (intrinsic)** và **quan trọng** của dữ liệu.
- Ví dụ:
 - **Frequent item set (Tập item phổ biến):** Các loại rau củ (cà rốt, hành tây, cần tây) thường được mua cùng nhau để nấu súp. **Đây là ví dụ kinh điển của "phân tích giỏ hàng".**
 - **Thứ tự không quan trọng.** {Sữa, Bánh mì} giống như {Bánh mì, Sữa}.
 - **Frequent sequences (Chuỗi phổ biến):** Các giai đoạn của bệnh cúm diễn ra theo một trình tự thời gian nhất định.
 - **Thứ tự rất quan trọng.** <Mua máy tính> rồi mới <Mua máy in> khác với <Mua máy in> rồi mới <Mua máy tính>.
 - **Frequent structures (Cấu trúc phổ biến):** Giao diện người dùng của một chương trình máy tính (ví dụ: menu "Spelling & Grammar" trong Microsoft Word) có một cấu trúc phân cấp cố định.
 - Không chỉ là thứ tự, mà còn là mối quan hệ phức tạp hơn, thường được biểu diễn dưới dạng đồ thị (graph). Ví dụ: cấu trúc phân tử hóa học.



Frequent item set



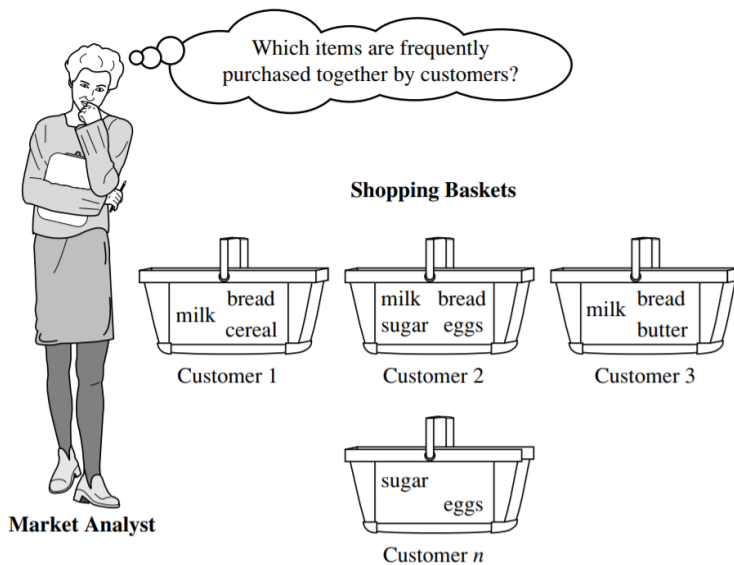
Frequent sequences



Frequent structures

6.2. Pattern Discovery (Khám phá Mẫu)

- Là quá trình khám phá ra các mẫu từ các tập dữ liệu khổng lồ (massive data sets).
- Khai phá mẫu đặc biệt hữu ích khi con người không thể tự mình rà soát hết dữ liệu. Các thuật toán sẽ tự động hóa quá trình này.
- Mục đích của nó là trả lời các câu hỏi kinh doanh hoặc khoa học, ví dụ:
 - "Những sản phẩm nào thường được mua cùng nhau?" (Ví dụ về phân tích giỏ hàng - Market Basket Analysis).
 - "Sau khi mua một chiếc iPad, khách hàng thường mua thêm gì?" (Ví dụ về phân tích chuỗi tuần tự).
 - Các trang thương mại điện tử như Amazon, Tiki sử dụng kỹ thuật này để gợi ý sản phẩm ("Những khách hàng đã mua sản phẩm này cũng đã mua...").
 - Netflix sử dụng nó để gợi ý phim dựa trên lịch sử xem của bạn và của những người dùng có sở thích tương tự.
 - Trong y học, nó có thể tìm ra mối liên hệ giữa các triệu chứng và một căn bệnh cụ thể.



• Why Is It Important?

- Là cơ sở cho nhiều tác vụ khai phá dữ liệu khác như:
 - Phân tích liên kết, tương quan, và nhân quả.
 - Khai phá mẫu tuần tự, cấu trúc.
 - Phân lớp (Classification) dựa trên mẫu.
 - Phân cụm (Clustering) dựa trên mẫu.
- Khai phá mẫu không phải là một mục tiêu cuối cùng, mà thường là một **bước trung gian quan trọng**. Ví dụ, sau khi tìm thấy các mẫu phổ biến, chúng ta có thể sử dụng chúng làm "đặc trưng" (features) để xây dựng một mô hình phân lớp chính xác hơn.
- Ứng dụng rộng rãi:
 - Phân tích giỏ hàng, tiếp thị chéo (cross-marketing), thiết kế danh mục sản phẩm, phân tích chiến dịch bán hàng.
 - **Ví dụ về tiếp thị chéo:** Nếu phát hiện ra mẫu {Tã, Bia} là phổ biến, siêu thị có thể đặt bia gần khu vực bán tã để tăng doanh số. Đây là một ví dụ kinh điển (dù có thể chỉ là giai thoại) trong ngành.
 - Phân tích nhật ký web (Web log analysis), phân tích chuỗi sinh học.
 - **Ví dụ về Phân tích chuỗi sinh học:** Tìm các đoạn gen (chuỗi DNA) phổ biến có thể giúp xác định các gen liên quan đến bệnh tật.
- Hỗ trợ nhiều loại dữ liệu: Dữ liệu không gian-thời gian (spatiotemporal), đa phương tiện (multimedia), chuỗi thời gian (time-series), và dữ liệu dòng (stream data).

6.3. Frequent Itemsets (Tập phổ biến)

6.3.1. Transactional Database (TDB) và các khái niệm liên quan

- Mỗi giao dịch có một mã định danh duy nhất gọi là **TID (Transaction ID)**.
- Một giao dịch chứa một tập hợp các item.
- TDB không chỉ giới hạn ở việc mua bán.** "Giao dịch" có thể là một tài liệu văn bản, và "item" là các từ trong tài liệu đó. "Giao dịch" có thể là một bệnh nhân, và "item" là các triệu chứng của họ.

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- Itemset:** A set of one or more items (tập các mặt hàng): $I = \{I_1, I_2, \dots, I_m\}$
- Giao dịch t:** tập các item, $t \subseteq I$
- CSDL giao dịch:** tập các t
 - $D = \{t_1, t_2, \dots, t_n\}$
 - $t_i = \{i_{i1}, i_{i2}, \dots, i_{ik}\}$ với $i_{ij} \in I$
- k-itemset:** An itemset containing k items (tập k mặt hàng): $X = \{x_1, \dots, x_k\}$
 - Ex. {Beer, Nuts, Diaper} is a 3-itemset

Tid	Items bought
1	Milk, Bread, Eggs
2	Bread, Sugar
3	Bread, Cereal
4	Milk, Bread, Sugar
5	Milk, Cereal
6	Bread, Cereal
7	Milk, Cereal
8	Milk, Bread, Cereal, Eggs
9	Milk, Bread, Cereal


Biến đổi CSDL về dạng nhị phân

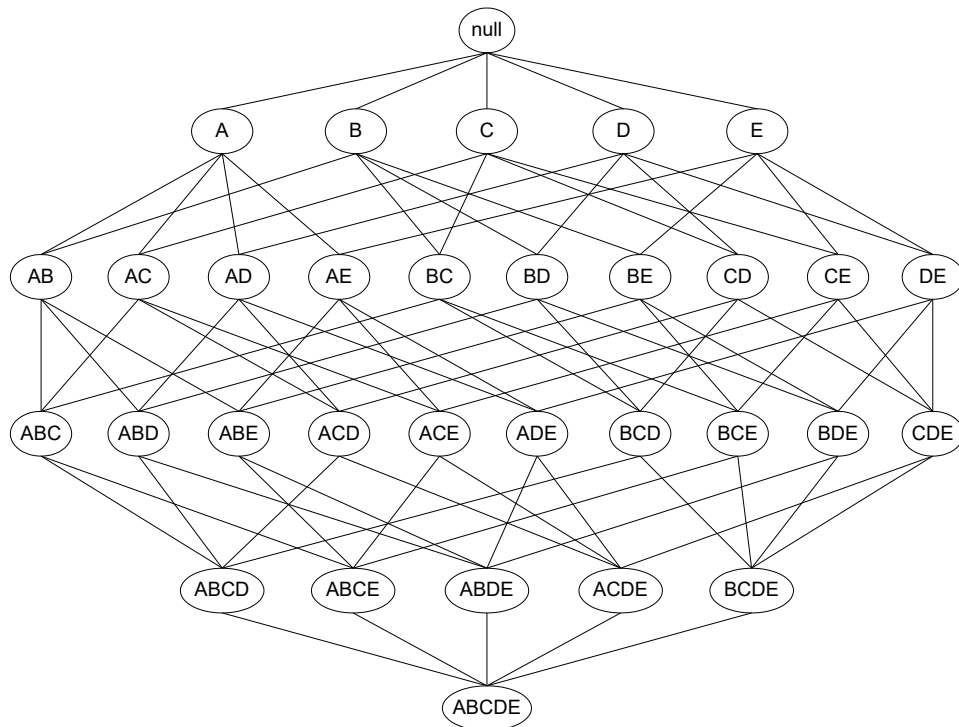
Items:
 A = Milk
 B = Bread
 C = Cereal
 D = Sugar
 E = Eggs

Tid	A	B	C	D	E
1	1	1	0	0	1
2	0	1	0	1	0
3	0	1	1	0	0
4	1	1	0	1	0
5	1	0	1	0	0
6	0	1	1	0	0
7	1	0	1	0	0
8	1	1	1	0	1
9	1	1	1	0	0

- **Absolute support (count) (Độ hỗ trợ tuyệt đối: số lần xuất hiện của mặt hàng)**
 - **sup(X)** = occurrences of an itemset X. **Ex:**
 - $\text{sup}\{\text{Beer}\} = 3$
 - $\text{sup}\{\text{Diaper}\} = 4$
 - $\text{sup}\{\text{Beer}, \text{Diaper}\} = 3$
 - $\text{sup}\{\text{Beer}, \text{Eggs}\} = 1$
- **Relative support (Độ hỗ trợ tương đối)**
 - **s{X}** = The fraction of transactions that contains X (i.e., the **probability that a transaction contains X**).
 - **s(X) = sup(X) / |TDB|**, trong đó |TDB| là tổng số giao dịch.
 - **Ex:**
 - $s\{\text{Beer}\} = 3/5 = 60\%$
 - $s\{\text{Diaper}\} = 4/5 = 80\%$
 - $s\{\text{Beer}, \text{Eggs}\} = 1/5 = 20\%$
- **Support (Độ hỗ trợ)** là thước đo mức độ phổ biến của một itemset. Đây là khái niệm quan trọng nhất trong khai phá mẫu liên kết. Một itemset có support cao có nghĩa là nó xuất hiện thường xuyên.
- Trong thực tế, chúng ta thường làm việc với độ hỗ trợ tương đối vì nó không phụ thuộc vào tổng số giao dịch. Điều này giúp so sánh kết quả giữa các tập dữ liệu có kích thước khác nhau.
- **Tính chất tập phổ biến: Tất cả các tập con của tập phổ biến đều là tập phổ biến**

6.3.2. Frequent Itemsets (Patterns)

- **Goal:** find combinations of items (itemsets) that occur **frequently**
- An itemset (or a pattern) X is **frequent** if the **support of X** is **no less than a minsup (minimum support) threshold σ** (ngưỡng này ta sẽ cung cấp, không phải do máy tính tạo ra)
 - Nếu minsup quá cao, chúng ta có thể bỏ lỡ nhiều mẫu thú vị.
 - Nếu minsup quá thấp, chúng ta sẽ tạo ra một số lượng khổng lồ các mẫu, nhiều trong số đó không có ý nghĩa và gây tốn kém tài nguyên tính toán.
- Let $\sigma = 50\%$ (σ : minsup threshold) for the given 5-transaction dataset
 - All the frequent 1-itemsets: Beer: 3/5 (60%); Nuts: 3/5 (60%); Diaper: 4/5 (80%); Eggs: 3/5 (60%)
 - All the frequent 2-itemsets: {Beer, Diaper}: 3/5 (60%)
 - All the frequent 3-itemsets? **None**
- **Number of possible itemsets: $M = 2^d$** , with **d (dimension)**: number of (distinct) items.

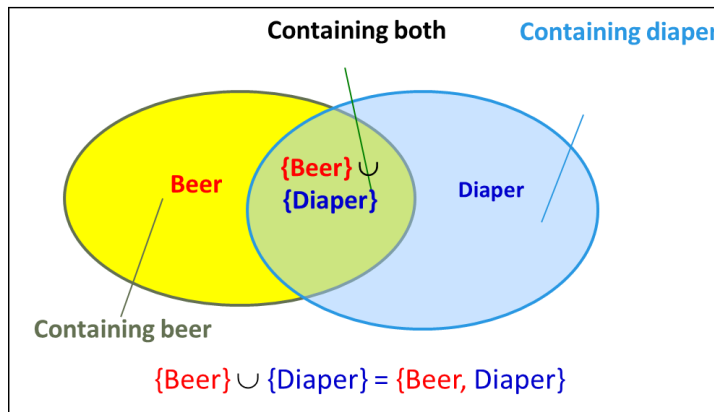


- We may need an efficient method to mine a complete set of frequent patterns

6.4. Association Rules (Luật kết hợp)

6.4.1. From Frequent Itemsets to Association Rules

- Luật kết hợp thường có ý nghĩa và dễ diễn giải hơn so với chỉ một tập item.
- Ví dụ: Diaper $\rightarrow$ Beer (Nếu mua Tã thì sẽ mua Bia).



- Note: $X \cup Y$: the union of two itemsets - The set contains both X and Y (các giao dịch chứa cả hai).
- Một luật kết hợp có dạng $X \rightarrow Y$, trong đó X và Y là các itemset và $X \cap Y = \emptyset$.
 - X được gọi là **vế trái (antecedent)** hoặc **giả thiết**.
 - Y được gọi là **vế phải (consequent)** hoặc **kết luận**.
 - Ý nghĩa: "Khi X xuất hiện trong một giao dịch, có khả năng cao Y cũng sẽ xuất hiện."
- Không phải tất cả các tập item phổ biến đều tạo ra các luật kết hợp mạnh. Chúng ta cần một thước đo khác để đánh giá sức mạnh của một luật.
- How do we compute the strength of an association rule $X \rightarrow Y$ (Both X and Y are itemsets)?

6.4.2. Hai thước đo để đánh giá sức mạnh của một luật kết hợp

- **Support (Độ hỗ trợ):**
 - Chính là độ hỗ trợ của itemset $X \cup Y$.
 - Cho biết tần suất của quy luật trong toàn bộ dữ liệu. Nó đo lường mức độ phổ biến của cả hai về xuất hiện cùng nhau.
 - Một quy luật có support thấp có thể chỉ là ngẫu nhiên.
 - $s\{\text{Diaper, Beer}\} = 3/5 = 60\%$.
- **Confidence (Độ tin cậy):**
 - Đo lường **xác suất có điều kiện $P(Y|X)$** .
 - Nó cho biết nếu một giao dịch chứa X, thì khả năng nó cũng chứa Y là bao nhiêu (*conditional probability* that a transaction containing X also contains Y).
 - Cho biết mức độ "đáng tin" của quy luật.
 - Confidence cao có nghĩa là khi về trái xảy ra, về phải gần như chắc chắn cũng xảy ra.
 - Công thức: **$c(X \rightarrow Y) = \text{sup}(X \cup Y) / \text{sup}(X)$** .
 - Ví dụ: $c(\text{Diaper} \rightarrow \text{Beer}) = \text{sup}\{\text{Diaper, Beer}\} / \text{sup}\{\text{Diaper}\} = 3 / 4 = 75\%$.
- In pattern analysis, we are often interested in those rules that dominate (chiếm ưu thế) the database, and these two metrics **ensure the popularity (tính phổ biến) and correlation (mối tương quan)** of X and Y.
- Một luật kết hợp tốt phải có cả **support cao** và **confidence cao**.
- **Lưu ý:** **$c(X \rightarrow Y)$ không nhất thiết bằng $c(Y \rightarrow X)$** .
 - $c(\text{Beer} \rightarrow \text{Diaper}) = \text{sup}\{\text{Beer, Diaper}\} / \text{sup}\{\text{Beer}\} = 3 / 3 = 100\%$.
 - Điều này có nghĩa là: **75% người mua tã cũng mua bia, nhưng 100% người mua bia (trong tập dữ liệu này) cũng mua tã.**

6.4.3. Mining Frequent Itemsets and Association Rules

- **Bài toán khai phá luật kết hợp:** Cho trước hai ngưỡng **minsup** và **minconf** (minimum confidence), tìm tất cả các luật $X \rightarrow Y$ (s, c) sao cho **$s(X \cup Y) \geq \text{minsup}$ và $c(X \rightarrow Y) \geq \text{minconf}$** .
- Quá trình này thường được chia thành hai bước:
 - **Tìm tất cả các tập item phổ biến (Frequent Itemset Generation):** Tìm tất cả các itemset có **support $\geq \text{minsup}$** . (Đây là bước tốn kém nhất).
 - **Tạo luật từ các tập item phổ biến (Rule Generation):** Với mỗi tập item phổ biến L, **tạo ra các luật $X \rightarrow (L-X)$ thỏa mãn minconf**.

Generate **high confidence** rules from each frequent itemset, where each rule is a **partitioning of a frequent itemset into Left-Hand-Side (LHS) and Right-Hand-Side (RHS)**

Ví dụ: Frequent itemset: {A,B,C,D}

E.g., Rule: AB $\rightarrow$ CD

All Candidate rules:

BCD $\rightarrow$ A, ACD $\rightarrow$ B, ABD $\rightarrow$ C, ABC $\rightarrow$ D,
CD $\rightarrow$ AB, BD $\rightarrow$ AC, BC $\rightarrow$ AD, AD $\rightarrow$ BC, AB $\rightarrow$ CD, AC $\rightarrow$ BD,
D $\rightarrow$ ABC, C $\rightarrow$ ABD, B $\rightarrow$ ACD, A $\rightarrow$ BCD

• **Ví dụ 1:** với **minsup = 50%** và **minconf = 50%**:

- Tập phổ biến {Beer, Diaper} có support 60%.
- Luật Beer → Diaper có confidence 100% (thỏa mãn). *Beer → Diaper (60%, 100%)*
- Luật Diaper → Beer có confidence 75% (thỏa mãn). *Diaper → Beer (60%, 75%)*

VD4:

Tid	Items
t1	A, B, C
t2	A, C
t3	A, D
t4	B, E, F

Minsupp = 50%, Minconf = 80%



Tập phổ biến	Supp
A	75%
B	50%
C	50%
A, C	50%

- **Luật A → C:**

$\text{supp}(A \rightarrow C) = \text{supp}(\{A\} \cup \{C\}) = 50\%$

$\text{conf}(A \rightarrow C) = \text{supp}(\{A\} \cup \{C\}) / \text{supp}(\{A\}) = 66.6\%$ (Loại)

- **Luật C → A:**

$\text{supp}(C \rightarrow A) = \text{supp}(\{C\} \cup \{A\}) = 50\%$

$\text{conf}(C \rightarrow A) = \text{supp}(\{C\} \cup \{A\}) / \text{supp}(\{C\}) = 100\%$ (Thỏa)

- **Quan sát:** Khai phá luật kết hợp và khai phá tập phổ biến là hai bài toán rất gần nhau. Cần có các phương pháp có thể mở rộng (scalable) để xử lý các tập dữ liệu lớn.

6.4.4. There Are Too Many Frequent Patterns!

• **Sự bùng nổ tổ hợp (combinatorial explosion).**

- Ví dụ: Một CSDL có 2 giao dịch: T₁: {a₁, ..., a₅₀}; T₂: {a₁, ..., a₁₀₀}
 - Với minsup = 1 (tức là mọi itemset xuất hiện ít nhất một lần đều được coi là phổ biến).
 - 1-itemsets: {a₁}: 2, {a₂}: 2, ..., {a₅₀}: 2, {a₅₁}: 1, ..., {a₁₀₀}: 1,
 - 2-itemsets: {a₁, a₂}: 2, ..., {a₁, a₅₀}: 2, {a₁, a₅₁}: 1 ..., ..., {a₉₉, a₁₀₀}: 1,
 - ..., ..., ..., ...
 - 99-itemsets: {a₁, a₂, ..., a₉₉}: 1, ..., {a₂, a₃, ..., a₁₀₀}: 1
 - 100-itemset: {a₁, a₂, ..., a₁₀₀}: 1
 - Tổng số itemset phổ biến là tổng của tất cả các tổ hợp có thể có từ 100 item, trừ đi tập rỗng, là $2^{100} - 1$.

$$\binom{100}{1} + \binom{100}{2} + \binom{100}{3} + \dots + \binom{100}{100} = 2^{100} - 1$$

- Đây là một con số khổng lồ, không thể tính toán hoặc lưu trữ.
- Đây là lý do tại sao việc đặt minsup quá thấp là không thực tế.
- Vấn đề này thúc đẩy sự ra đời của các kỹ thuật nén thông tin, nhằm chỉ giữ lại những mẫu "cốt lõi" nhất mà từ đó có thể suy ra các mẫu khác.

6.4.4.1. Closed Patterns (Mẫu đóng)

- Một itemset X là một mẫu đóng **nếu nó phổ biến**, và **không tồn tại một siêu mẫu** (super-pattern) $Y \supset X$ nào có cùng độ hỗ trợ với X .
- Mẫu đóng là itemset "lớn nhất" trong một nhóm các itemset có cùng support.
- Ví dụ:**
 - Let Transaction DB TDB₁: T₁: {a₁, ..., a₅₀}; T₂: {a₁, ..., a₁₀₀}
 - Suppose $minsup = 1$. How many closed patterns does TDB₁ contain? Two: P₁: "{a₁, ..., a₅₀}: 2"; P₂: "{a₁, ..., a₁₀₀}: 1"

VD3: Minsupp = 2

- {A, B}: tập phổ biến đóng
- {A, B, D}: tập phổ biến đóng
- {A, B, C}: tập phổ biến đóng

Tid	Items
t1	A, B, C
t2	A, B, C
t3	A, B, D
t4	A, B, D
t5	C, E, F

- Lợi ích:** Mẫu đóng là một dạng **nén không mất mát (lossless compression)**. Từ tập hợp các mẫu đóng và support của chúng, ta có thể suy ra chính xác support của tất cả các tập item phổ biến khác.
 - Ví dụ: support của {a₂, ..., a₄₀} là bao nhiêu? Vì {a₂, ..., a₄₀} là tập con của {a₁, ..., a₅₀}, và không có mẫu đóng nào khác chứa nó và có support 2, nên support của nó cũng là 2.
- Khai phá mẫu đóng hiệu quả hơn nhiều so với **khai phá tất cả** các mẫu phổ biến vì số lượng mẫu đóng thường nhỏ hơn đáng kể.
- Từ tập mẫu đóng, ta có thể tạo ra tất cả các luật kết hợp mà không mất thông tin.

6.4.4.2. Max-Patterns (Mẫu cực đại)

- Một itemset X là một mẫu cực đại **nếu nó phổ biến**, và không tồn tại một siêu mẫu phổ biến nào chứa nó.
- Mẫu cực đại là các itemset phổ biến "dài nhất", không thể mở rộng thêm mà vẫn giữ được tính phổ biến.
- Ví dụ với CSDL TDB₁: T₁: {a₁, ..., a₅₀}; T₂: {a₁, ..., a₁₀₀} ($minsup = 1$):
 - Chỉ có một mẫu cực đại: P: "{a₁, ..., a₁₀₀}" với support 1.

VD2: Minsupp = 2

- {B, C, D, E}: tập phổ biến tối đại
- {A, C, D}: tập phổ biến tối đại
- {B, C, D}: không là tập phổ biến tối đại

Tid	Items
t1	A, B, C, D, E
t2	B, C, D, E
t3	A, C, D, F

- Số lượng mẫu cực đại thậm chí còn ít hơn số lượng mẫu đóng. Việc khai phá chúng rất nhanh.
- Nhược điểm:** Mẫu cực đại là một dạng **nén mất mát (lossy compression)**. Chúng ta chỉ biết rằng các tập con của một mẫu cực đại (ví dụ {a₁, ..., a₄₀}) là phổ biến, nhưng chúng ta **không biết support chính xác** của chúng là bao nhiêu. Nó có thể là 1 hoặc 2.
 - Do mất thông tin về support của các tập con, mẫu cực đại không phù hợp cho các ứng dụng cần tính toán độ tin cậy của luật kết hợp một cách chính xác.
- Trong nhiều ứng dụng, khai phá **mẫu đóng (closed patterns)** là lựa chọn tốt hơn vì nó cân bằng giữa việc giảm số lượng mẫu và bảo toàn thông tin.

6.5. Bài toán khám phá Luật kết hợp

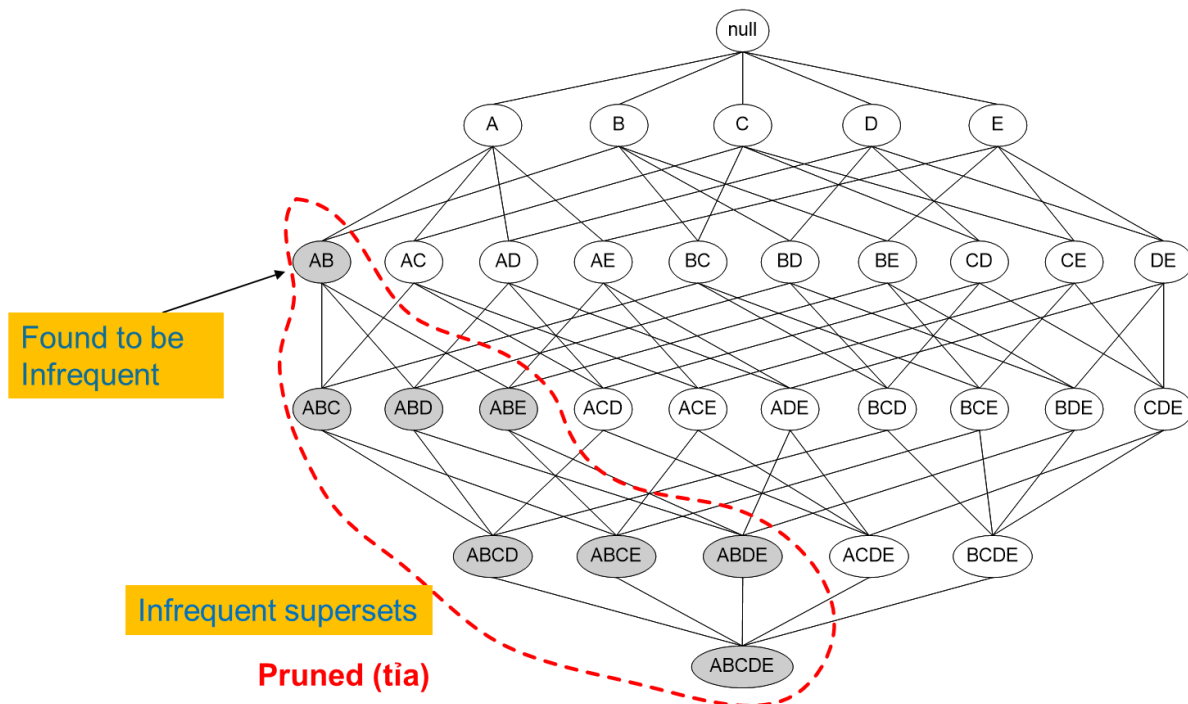
- Đưa về bài toán tìm tập phổ biến.
- Ba hướng tiếp cận chính:
 - **Level-wise, join-based (Theo từng mức, dựa trên phép kết):** Thuật toán **Apriori**, hoạt động theo chiều rộng (breadth-first), tìm tất cả các k-itemset phổ biến trước khi tìm (k+1)-itemset.
 - **Vertical data format (Định dạng dữ liệu dọc):** Thuật toán **Eclat**, thay đổi cách biểu diễn dữ liệu từ ngang (TID → list of items) sang dọc (Item → list of TIDs) để tăng tốc độ tính toán support bằng phép giao tập hợp.
 - **Frequent pattern projection and growth (Chiều và phát triển mẫu phổ biến):** Thuật toán **FP-Growth**, là một cách tiếp cận theo chiều sâu (depth-first), nén cơ sở dữ liệu vào một cấu trúc cây (FP-tree) và khai phá trực tiếp trên cây đó, tránh việc tạo ra và quét các ứng viên lặp đi lặp lại.

6.5.1. Nguyên lý Apriori (Apriori Principle)/ The Downward Closure Property

- **Cách tiếp cận:** Tạo và thử nghiệm các tập ứng viên
- **Phát biểu:** Mọi tập con của một tập item phổ biến cũng phải là một tập phổ biến.
- **Hệ quả - Nguyên tắc loại bỏ Apriori/ Reducing Number of Candidates:** Nếu một tập item không phổ biến, thì mọi siêu tập (tập bao) của nó cũng không thể phổ biến.
 - Support của một itemset không bao giờ vượt quá support của các tập con của nó.

$$\forall X, Y: X \subseteq Y \Rightarrow s(X) \geq s(Y)$$

- This is known as the **anti-monotone (chống đơn điệu)** property of support.
- **Anti-monotone** có nghĩa là khi một thuộc tính (ở đây là **kích thước của itemset**) **tăng lên**, thì một giá trị khác liên quan đến nó (ở đây là **support**) **sẽ không tăng (tức là nó sẽ giảm hoặc giữ nguyên)**.
- **Ví dụ:** $X = \{\text{Sữa, Bánh mì}\}$ là tập con của $Y = \{\text{Sữa, Bánh mì, Trứng}\}$.
 - Hãy xét một giao dịch (một hóa đơn mua hàng) bất kỳ. Nếu giao dịch này **chứa itemset Y**, điều đó có nghĩa là nó chứa tất cả các item của Y (ví dụ: chứa cả Sữa, Bánh mì, và Trứng) → Giao dịch này **chắc chắn cũng phải chứa X**. (Nếu hóa đơn có cả Sữa, Bánh mì, Trứng thì chắc chắn nó phải có Sữa và Bánh mì).
 - Nghĩa là: **Tập hợp các giao dịch chứa Y** là một **tập hợp con** của **tập hợp các giao dịch chứa X**.
 - Vì một tập hợp con không bao giờ có thể có nhiều phần tử hơn tập hợp cha của nó, nên **số lượng giao dịch chứa Y** luôn **nhỏ hơn hoặc bằng** số lượng giao dịch chứa X.
 - Từ đó suy ra: **$\text{count}(Y) \leq \text{count}(X)$** .
 - **Mà $s(A) = (\text{Số giao dịch chứa A}) / (\text{Tổng số giao dịch})$** . Chia cả hai vế cho tổng số giao dịch (một số dương), bất đẳng thức không đổi chiều: $s(Y) \leq s(X)$, hay viết ngược lại là **$s(X) \geq s(Y)$** .



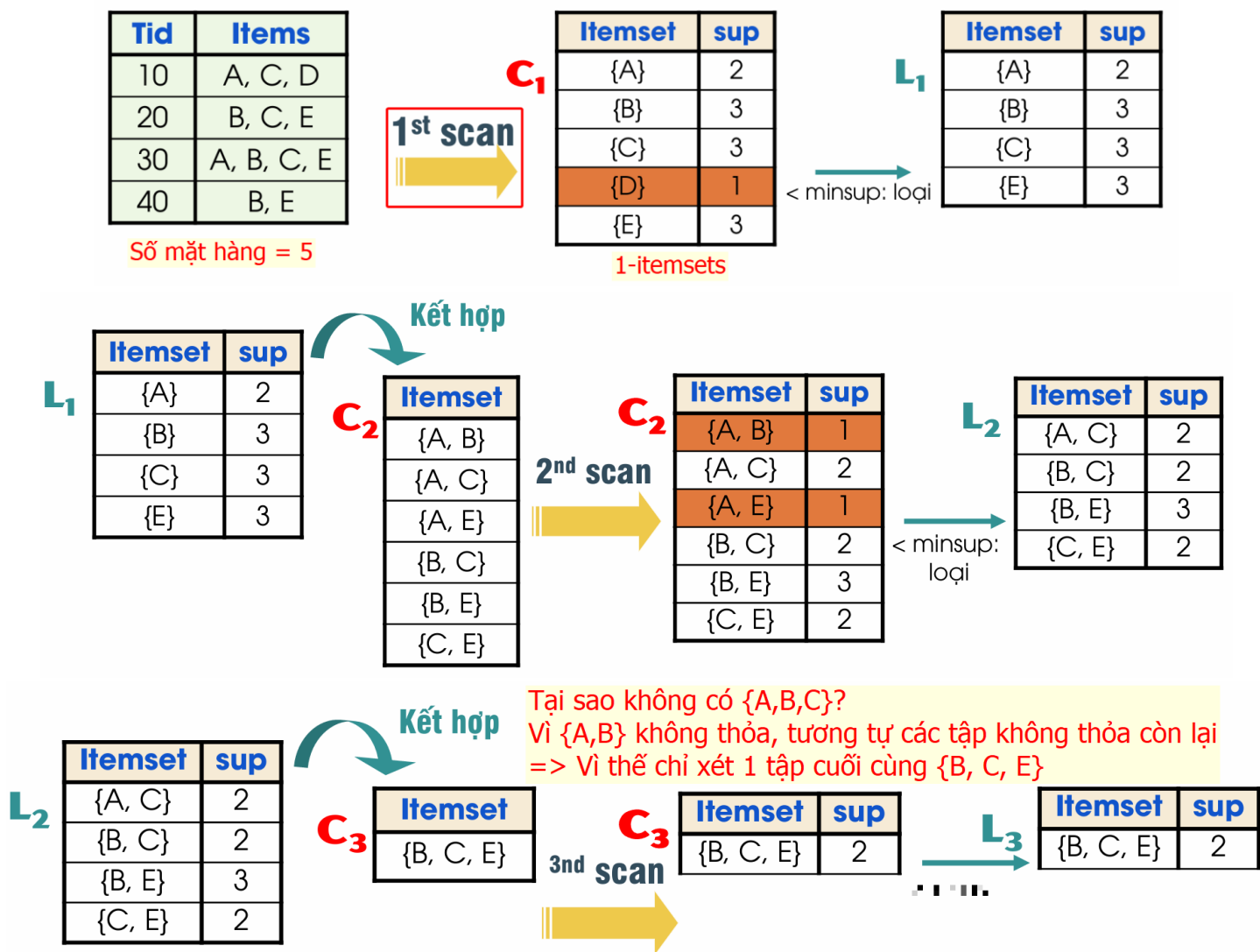
- **Ví dụ:** Nếu {beer, diaper, nuts} là phổ biến, thì {beer, diaper} chắc chắn cũng phổ biến. Ngược lại, nếu {beer, coffee} không phổ biến, thì không cần phải kiểm tra {beer, coffee, diaper} vì nó chắc chắn cũng không phổ biến.
- **Ứng dụng:** Nguyên lý này cung cấp một "con dao sắc bén để cắt tỉa" (a sharp knife for pruning) không gian tìm kiếm. Thay vì phải kiểm tra tất cả $2^n - 1$ itemset, chúng ta có thể loại bỏ hàng loạt các ứng viên không có tiềm năng.
- Nếu không có nó, chúng ta sẽ phải đếm support cho mọi itemset → vấn đề bùng nổ tổ hợp.
- Hầu hết các thuật toán khai phá tập phổ biến đều dựa trên nguyên lý này để tối ưu hóa quá trình tìm kiếm.
- **Điểm yếu chính của Apriori:** Nó đòi hỏi phải quét toàn bộ cơ sở dữ liệu nhiều lần (k-lần nếu mẫu dài nhất có k item). Với các CSDL lớn, việc này rất tốn kém.
- **Nút thắt cổ chai (bottleneck):** Việc tạo ra một số lượng lớn các ứng viên ở mỗi bước cũng có thể tốn nhiều bộ nhớ và thời gian xử lý.

6.5.1.1. Các bước của thuật toán Apriori (A Candidate Generation & Test Approach)

- Tìm tất cả các tập phổ biến **1-itemset**.
- **Bước lặp (Repeat):**
 - Tạo các tập ứng viên k-item từ **các tập phổ biến 1-item**
 - **Kiểm tra (Test):** Quét CSDL để **đếm support** của các ứng viên này.
 - **Cắt tỉa (Prune):** Loại bỏ các ứng viên không đạt minsup để có được tập (k+1)-itemset phổ biến.
 - Tăng k lên 1.
- **Dừng lại (Until):** Khi không thể tạo thêm ứng viên hoặc không có ứng viên nào phổ biến.
- **Kết quả:** Trả về tất cả các tập item phổ biến đã tìm được ở mỗi bước.

6.5.1.2. Example

- Tìm tập phổ biến, **minsup = 2**



- Các tập phổ biến: {A}, {B}, {C}, {E}, {A, C}, {B, C}, {B, E}, {C, E}, {B, C, E} → tổng cộng 9 tập phổ biến.

6.5.1.3. Improvements and Alternatives

- Thách thức:
 - Phải duyệt CSDL nhiều lần
 - Số lượng tập ứng viên rất lớn
 - Thực hiện việc tính độ phổ biến nhiều
 - VD: Tìm tập phổ biến 100 items:
 - Số lần duyệt CSDL 100.
 - Số lượng tập ứng viên $2^{100}-1$

- **Các phương pháp cải tiến và thay thế cho thuật toán Apriori gốc:**

- **Giảm số lần quét CSDL:**

- **Partitioning (Phân hoạch) (Chia để trị):**

- Bất kỳ itemset nào phổ biến trên toàn bộ CSDL (TDB) thì nó phải phổ biến trong ít nhất một phân hoạch (TDB_i).
 - Chia CSDL thành các phần nhỏ (các phân hoạch), tìm mẫu phổ biến cục bộ trong từng phân hoạch, sau đó hợp nhất.
 - Chỉ cần quét toàn bộ CSDL 2 lần.

- **Lần quét 1:**

- Chia TDB thành các phân hoạch nhỏ hơn sao cho mỗi phân hoạch có thể được xử lý trong bộ nhớ chính.
 - Tìm các tập item **phổ biến cục bộ** (local frequent patterns) trong mỗi phân hoạch.

- **Lần quét 2:**

- Hợp nhất kết quả: Tập các ứng viên toàn cục là hợp của tất cả các tập phổ biến cục bộ.
 - Quét toàn bộ TDB một lần nữa để đếm support thực sự của các ứng viên toàn cục này và tìm ra các tập phổ biến toàn cục.

- **Ưu điểm lớn:** Giảm số lần quét CSDL từ k lần xuống chỉ còn 2 lần, một cải tiến đáng kể.
 - Kỹ thuật này rất hiệu quả cho các CSDL cực lớn không thể vừa trong bộ nhớ.

- **Dynamic itemset counting (Đếm itemset động):** Thêm các ứng viên mới vào giữa quá trình quét.

- **Thu hẹp số lượng ứng viên:**

- **Hashing (Băm):** Sử dụng bảng băm để ước tính support của các itemset và loại bỏ sớm những ứng viên không có khả năng.

- Băm các tập ứng viên k-item vào các giỏ
 - Tập ứng viên k-item tương ứng có độ phổ biến < minsup sẽ bị loại

- **Pruning by support lower bounding (Cắt tỉa bằng cận dưới support).**

- **Sampling (Lấy mẫu):** Chọn mẫu từ CSDL lớn và tìm tập phổ biến trên mẫu, kiểm tra bao đóng của các tập phổ biến

- **Sử dụng các cấu trúc dữ liệu đặc biệt:**

- **Tree projection (Chiều cây):** FP-Growth là một ví dụ.
 - H-miner, Hpecube decomposition.

- **Direct Hashing and Pruning (DHP)**

- **Hashing:** $v = \text{hash}(\text{itemset})$

- **1st scan:** When counting the 1-itemset, hash 2-itemset to calculate the bucket count

- **Example:** At the 1st scan of TDB, count 1-itemset, and hash 2-itemsets in the transaction to its bucket

- {ab, ad, ce}
 - {bd, be, de}
 - ...

- At the end of the first scan,

- if $\text{minsup} = 80$, remove ab, ad, ce, since $\text{count}\{ab, ad, ce\} < 80$

V might be same for different itemset

Itemsets	Count
{ab, ad, ce}	35
{bd, be, de}	298
.....	...
{yz, qs, wt}	58

Hash Table

Check the minsup

A key observation: A k -itemset cannot be frequent if its corresponding hashing bucket count is below the minsup threshold

DHP (Direct Hashing and Pruning): (J. Park, M. Chen, and P. Yu, SIGMOD'95)

- **Nhược điểm:** Có thể có "xung đột băm" (hash collision), tức là nhiều itemset khác nhau được băm vào cùng một xô. Điều này có thể làm cho số đếm của xô lớn hơn minsup ngay cả khi không có itemset nào trong đó là phổ biến. Tuy nhiên, kỹ thuật này không bao giờ loại bỏ sai một itemset phổ biến, nó chỉ có thể giữ lại một vài itemset không phổ biến (false positives), điều này vẫn tốt hơn là không tìm gì cả.

▪ Exploring Vertical Data Format: ECLAT (Equivalence Class Transformation)

- A depth-first search algorithm using set intersection, khác với Apriori (breadth-first).
- **ECLAT** sử dụng định dạng dữ liệu dọc thay vì định dạng ngang (giao dịch). Thay vì lưu $TID \rightarrow \{Items\}$, ta lưu $Item \rightarrow \{TIDs\}$ (gọi là Tid-List).
- Support của một itemset X chính là độ dài (số lượng phần tử) của Tid-List của nó. $sup(X) = |t(X)|$.
- **Khai phá:**
 - Để tìm support của một 2-itemset $\{A, B\}$, ta chỉ cần thực hiện phép **giao** giữa Tid-List của A và Tid-List của B: $t(\{A,B\}) = t(A) \cap t(B)$.
 - Quá trình này được lặp lại: để tìm support của $\{A,B,C\}$, ta giao $t(\{A,B\})$ với $t(C)$.
- **Diffset:** Một cải tiến để tăng tốc. Thay vì lưu toàn bộ Tid-List, ta chỉ lưu sự khác biệt (diffset) giữa Tid-List của itemset con và itemset cha.
- **Ưu điểm của ECLAT:**
 - Không cần quét CSDL nhiều lần.
 - Việc tính support rất nhanh bằng các phép toán trên tập hợp, thường được tối ưu ở cấp độ phần cứng.
- **Nhược điểm:** Các Tid-List trung gian có thể rất lớn, tốn nhiều bộ nhớ, đặc biệt là với các item phổ biến.

A transaction DB in Horizontal Data Format

Tid	Itemset
10	a, c, d, e
20	a, b, e
30	b, c, e

The transaction DB in Vertical Data Format

Item	TidList
a	10, 20
b	20, 30
c	10, 30
d	10
e	10, 20, 30

$t(e) = \{T_{10}, T_{20}, T_{30}\};$
 $t(a) = \{T_{10}, T_{20}\};$
 $t(ae) = \{T_{10}, T_{20}\}$

6.5.2. Thuật toán FP-Growth

- Chúng ta có thể phát triển một thuật toán tìm kiếm theo chiều sâu không? Khi **đã tìm thấy một itemset phổ biến p**, liệu chúng ta có thể giới hạn việc **tìm kiếm các siêu mẫu của p chỉ trong các giao dịch chứa p không?**
- Có, và tư duy này dẫn đến cách tiếp cận **FP-Growth**.
- Ý tưởng cốt lõi là: để tìm các mẫu bắt đầu bằng p, chúng ta chỉ cần xem xét "cơ sở dữ liệu con" (conditional database) bao gồm các phần còn lại của các giao dịch chứa p.
- **FP-Growth Algorithm:**
 1. Step 1: Count the frequency of occurrence of each item.
 2. Step 2: Sort the items in descending order of their frequency.
 3. Step 3: Construct the **FP-Tree**.
 4. Step 4: Mine the frequent itemsets based on the **FP-Tree**.

- Thuật toán này minh họa thông qua ví dụ sau:

TID	Items in the Transaction
100	{f, a, c, d, g, i, m, p}
200	{a, b, c, f, l, m, o}
300	{b, f, h, j, o, w}
400	{b, c, k, s, p}
500	{a, f, c, e, l, p, m, n}

- Let $\text{min_support} = 3$
- Scan DB once, find single item frequent pattern (Tập phổ biến 1-item thỏa minsupp). Sort these frequent items in frequency **descending order**.
 → **F-list = f-c-a-b-m-p**
 - Mặc dù a, b, m, p có cùng minsup, tuy nhiên ta phải xếp thứ tự của chúng tuân theo thứ tự chúng xuất hiện trong tập giao dịch gốc. Chẳng hạn trong giao dịch TID=100, m xuất hiện trước p nên m phải đứng trước p.
- Scan DB again, use the ordered frequent itemlist for each transaction to construct an FP-tree

Itemset	sup
f	4
c	4
a	3
b	3
m	3
p	3

TID	Items in the Transaction	Ordered, frequent itemlist
100	{f, a, c, d, g, i, m, p}	f, c, a, m, p
200	{a, b, c, f, l, m, o}	f, c, a, b, m
300	{b, f, h, j, o, w}	f, b
400	{b, c, k, s, p}	c, b, p
500	{a, f, c, e, l, p, m, n}	f, c, a, m, p

- Xây dựng cây FP-tree (Frequent Pattern Tree):
 - Cấu trúc:
 - FP-tree:** Một cấu trúc cây nén, mỗi nút biểu diễn một item và có một bộ đếm.
 - Header Table:** Mỗi mục trong bảng tương ứng với một item phổ biến, chứa support của item đó và một con trỏ đến nút đầu tiên của item đó trong cây. Các nút cùng item được liên kết với nhau thành một danh sách liên kết.
- Quá trình xây dựng:
 - Khởi tạo cây với một nút gốc (root) rỗng.
 - Lần lượt duyệt qua từng giao dịch đã được sắp xếp trong bảng (cột Ordered, frequent itemlist).
 - Với mỗi giao dịch, đi từ gốc, tạo ra một nhánh tương ứng với các item trong giao dịch.
 - Nếu một nút đã tồn tại trên đường đi, chỉ cần tăng bộ đếm của nó thêm 1. **Nếu chưa, tạo nút mới với bộ đếm là 1.**
 - Cập nhật các con trỏ trong Header Table.

Xét giao dịch đầu tiên chỉ còn các item phổ biến đã được sắp xếp giảm dần theo support

TID	Ordered, frequent itemlist
100	<i>f, c, a, m, p</i>

After inserting the 1st frequent Itemlist: "f, c, a, m, p"

Item	Frgncy	hdr
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	

Header Table



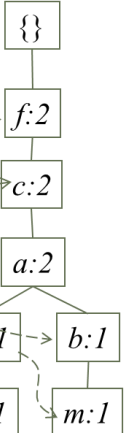
Xét giao dịch tiếp theo:

TID	Ordered, frequent itemlist
200	<i>f, c, a, b, m</i>

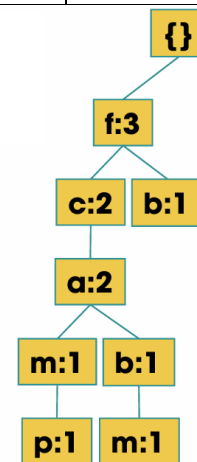
- Tiếp tục tạo cây từ TID=200, ta thấy f-c-a ban đầu đã hình thành ở TID=100, vì thế tăng bộ đếm của chúng lên 1 (là 2).
- Khác với TID=100, sau f-c-a thì trong TID=200 nó là b-m thay vì m-p, vì thế ta phải rẽ sang một nhánh mới (như hình) và cập nhật bộ đếm của b-m là 1.

After inserting the 2nd frequent itemlist "f, c, a, b, m"

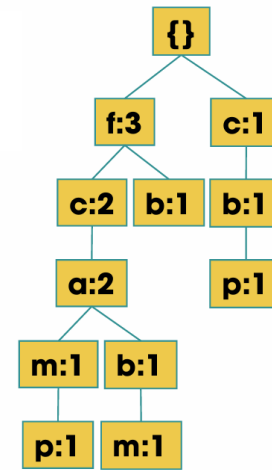
itm	hdr
f	
c	
a	
b	
m	
p	



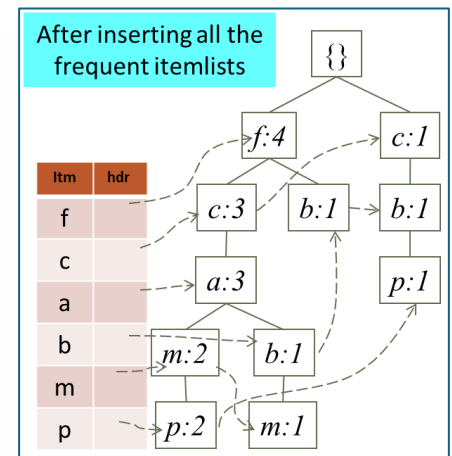
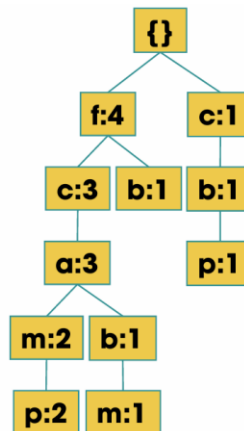
Tid	Items	Frequent-items (orderd)
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o, w}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}



Tid	Items	Frequent-items (orderd)
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o, w}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}



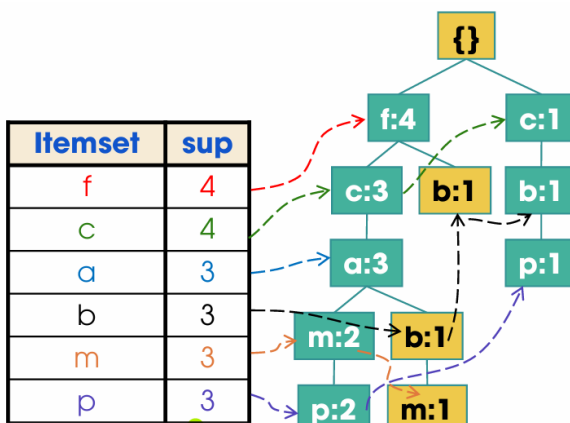
Tid	Items	Frequent-items (orderd)
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o, w}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}



- Duyệt cây FP theo kết nối của mỗi mẫu phổ biến → Conditional database of each pattern. Sau đó tìm các tập phổ biến dựa vào minsup.

- Xét mẫu p: Đi từ dưới lên:

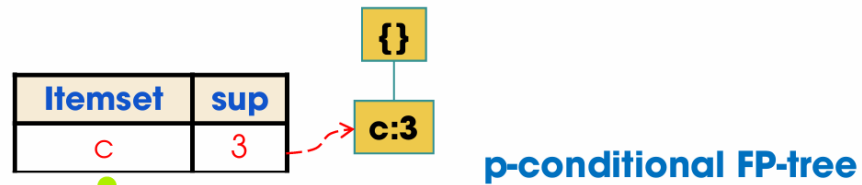
- p-m-a-c-f, lấy số đếm min của chuỗi này → 2 → fcam:2 (nghĩa là f:2, c:2, a:2, m:2)
- p-b-c, min = 1 → cb:1 (c:1, b:1)



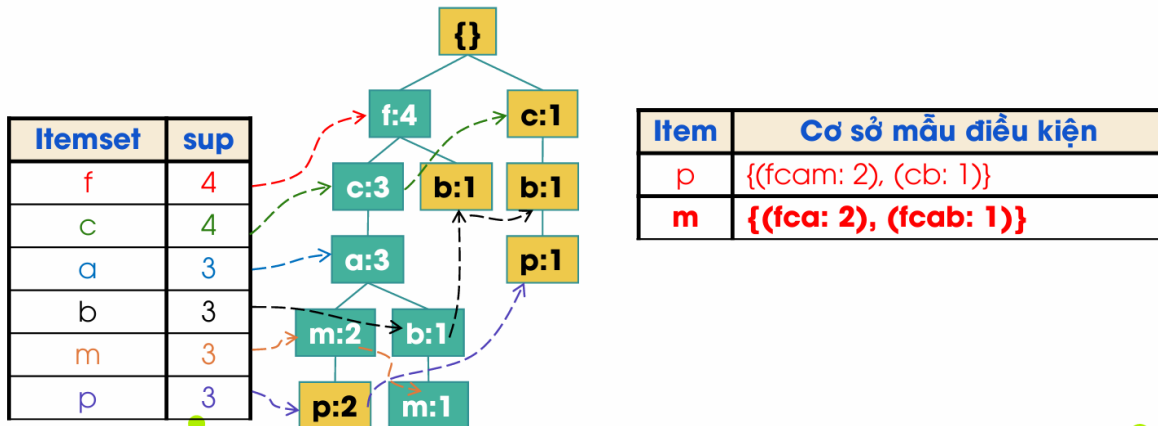
Item	Cơ sở mẫu điều kiện
p	{(fcam: 2), (cb: 1)}

- Với cơ sở mẫu điều kiện của p: {(fcam: 2), (cb: 1)}
 - Đếm số lượng mỗi mẫu trong cơ sở mẫu: (f: 2, c: 3, a: 2, m: 2), (c:3, b: 1) với Minsupp = 3
 - => c:3 phổ biến trên cơ sở mẫu điều kiện của p (vì thỏa $\geq$ minsup)

- Thiết lập cây FP cho tập phổ biến của cơ sở mẫu điều kiện của p

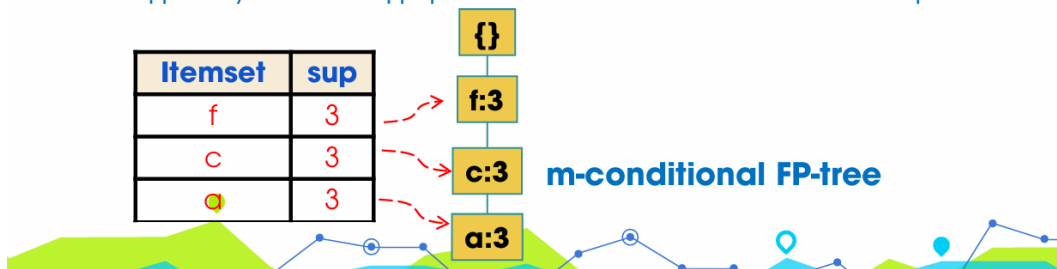


- Tập mẫu phổ biến liên quan đến p: **p, cp**

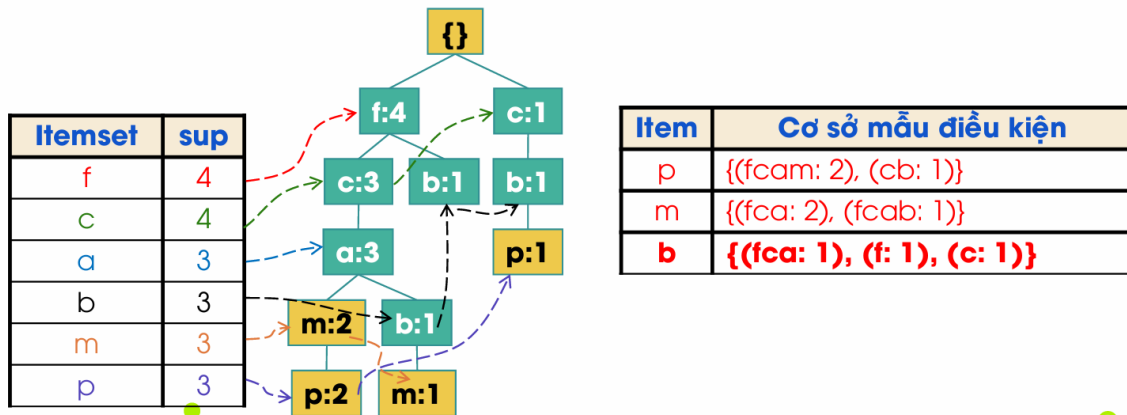


Với cơ sở mẫu điều kiện của **m**: **{{(fca: 2), (fcab: 1)}}**

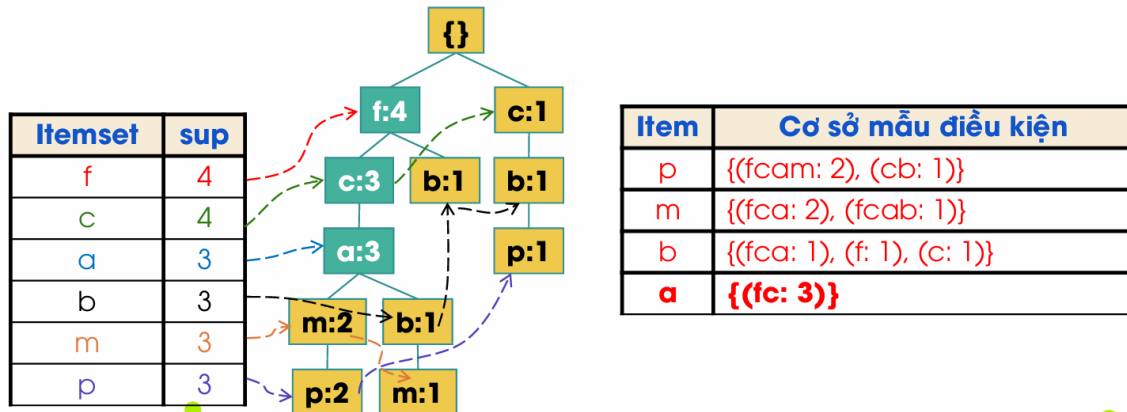
- Đếm số lượng mỗi mẫu trong cơ sở mẫu:
 - (f: 3, c: 3, a: 3), (f: 3, c: 3, a: 3, b: 1) với Minsupp = 3
 - => (f: 3, c: 3, a: 3) phổ biến trên cơ sở mẫu điều kiện của **m**
- Thiết lập cây FP cho tập phổ biến của cơ sở mẫu điều kiện của **m**



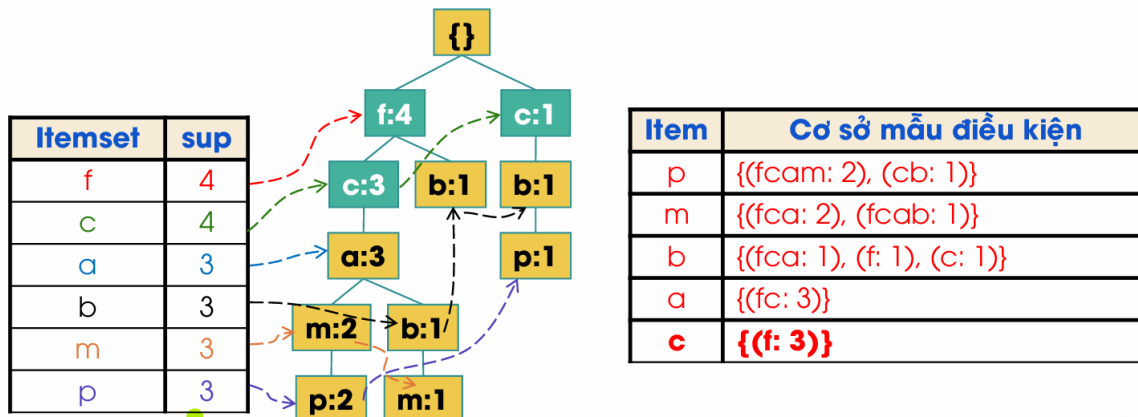
- Liệt kê tất cả các tổ hợp của f:3, c:3, a:3, chính là Tập mẫu phổ biến liên quan đến m:
 - m**
 - fm, cm, am**
 - fcm, fam, cam**
 - fcam.**



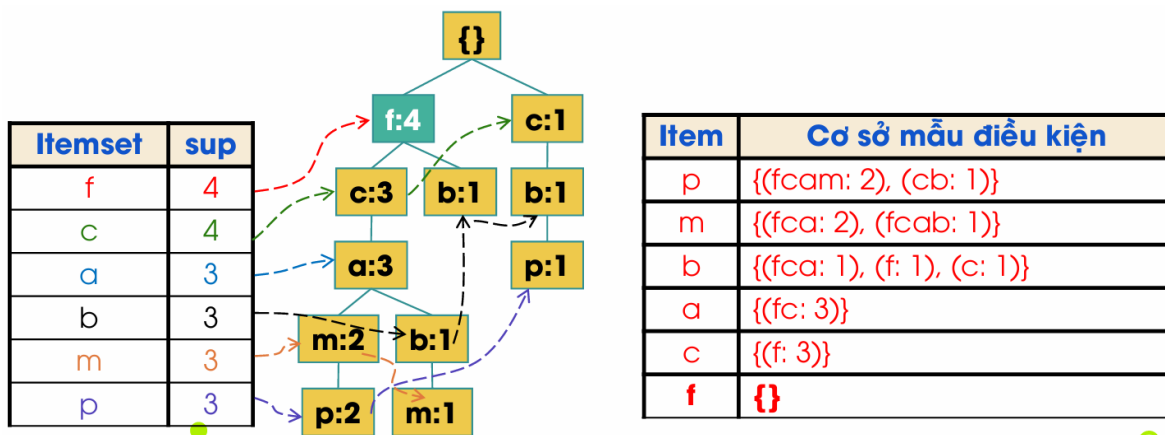
- Dễ thấy không có mẫu nào phổ biến trên cơ sở mẫu điều kiện của b.
- Tập mẫu phổ biến liên quan đến b: **b**



- (f:3, c:3) phổ biến trên cơ sở mẫu điều kiện của a.
- Tập mẫu phổ biến liên quan đến a: **a, fa, ca, fca.**



- (f:3) phổ biến trên cơ sở mẫu điều kiện của c.
- Tập mẫu phổ biến liên quan đến c: **c, fc.**



- Không có mẫu nào phổ biến trên cơ sở mẫu điều kiện của f.
- Tập mẫu phổ biến liên quan đến f: **f**.

Tổng kết:

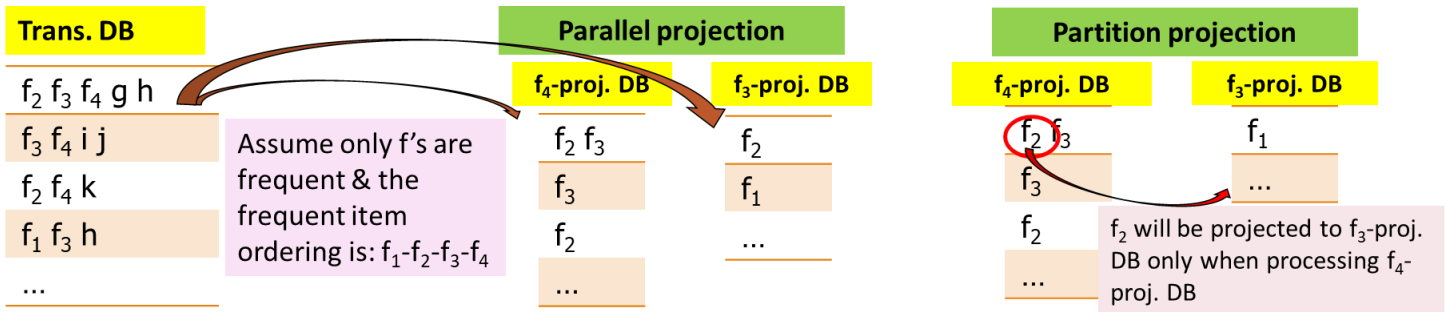
Item	sup	Cây FP điều kiện	Tập phổ biến
p	4	{{(c: 3)} p	p: 4, cp: 3
m	4	{{(f: 3, c: 3, a: 3)} m	m: 4, fm: 3, cm: 3, am: 3, fcm: 3, fam: 3, cam: 3, fcam: 3
b	3	{}	b: 3
a	3	{{(f: 3, c: 3)} a	a: 3, fa: 3, ca: 3, fca: 3
c	3	{{(f: 3)} c	c: 3, fc: 3
f	3	{}	f: 3

Essence of frequent pattern growth (FP-Growth) methodology

- Find frequent single items and partition the database based on each such single item pattern (Tìm các 1-item phổ biến và phân hoạch không gian tìm kiếm dựa trên chúng).
- Recursively grow frequent patterns by doing the above for each *partitioned database* (also called the pattern's *conditional database*) (Phát triển các mẫu dài hơn một cách đệ quy bằng cách làm việc trên các CSDL có điều kiện).
- To facilitate efficient processing, an efficient data structure, FP-tree, can be constructed (Sử dụng cấu trúc FP-tree hiệu quả để nén CSDL và hỗ trợ quá trình khai phá).
- Trong hầu hết các trường hợp, FP-Growth nhanh hơn đáng kể so với Apriori, đặc biệt là khi dữ liệu có nhiều mẫu dài và minsup tương đối thấp.

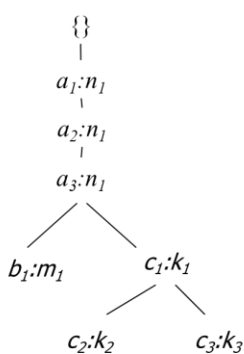
6.5.2.1. Điều gì xảy ra nếu FP-tree không vừa trong bộ nhớ?

- Không xây dựng một FP-tree khổng lồ. Thay vào đó, sử dụng kỹ thuật **chiếu cơ sở dữ liệu (database projection)**.
- Parallel Projection (Chiếu song song):**
 - Với mỗi item phổ biến f_i , tạo ra một CSDL chiếu (projected DB) chỉ chứa các giao dịch có f_i .
 - Ưu điểm: Các CSDL chiếu này có thể được xử lý song song.
 - Nhược điểm: Tốn nhiều không gian lưu trữ vì các giao dịch có thể bị lặp lại ở nhiều CSDL chiếu.
- Partition Projection (Chiếu phân hoạch):**
 - Xử lý các item theo thứ tự.
 - Tạo CSDL chiếu cho f_4 , xử lý nó. Sau đó, phần dữ liệu chưa được xử lý sẽ được chuyển cho CSDL chiếu của f_3 , và cứ thế.
 - Ưu điểm: Tiết kiệm không gian hơn.
- Ý tưởng cơ bản vẫn là "chia để trị", nhưng thay vì chia trên cây FP-tree, ta chia trực tiếp trên cơ sở dữ liệu.



6.5.2.2. CLOSET+: Mining Closed Itemsets by Pattern-Growth

- một biến thể của FP-Growth để khai phá trực tiếp các **mẫu đóng (closed itemsets)**.
- Nếu một cây FP-tree có điều kiện chỉ có một nhánh, các item trên nhánh đó có thể được "gộp" lại với nhau nếu chúng luôn xuất hiện cùng nhau (cùng support).



- Efficient, *direct* mining of closed itemsets
- Intuition:
 - If an FP-tree contains a single branch as shown left
 - "a₁, a₂, a₃" should be merged
- Itemset merging: If Y appears in every occurrence of X, then Y is merged with X
 - d-proj. db: {ac~~ef~~, ac~~f~~} → acfd-proj. db: {e}
 - Final closed itemset: acfd:2
- There are many other tricks developed
 - For details, see J. Wang, et al., "CLOSET+: Searching for the Best Strategies for Mining Frequent Closed Itemsets", KDD'03

TID	Items
1	acdef
2	abe
3	cefg
4	acdf

Let minsupport = 2

a:3, c:3, d:2, e:3, f:3

F-List: a-c-e-f-d

6.5.2.3. Mở rộng FP-Growth

- Mining closed frequent itemsets and max-patterns: CLOSET(DMKD'00), FPclose, and FPMax (Grahne & Zhu, Fimi'03)
- Mining sequential patterns: PrefixSpan (ICDE'01), CloSpan (SDM'03), BIDE (ICDE'04)
- Mining graph patterns: gSpan(ICDM'02), CloseGraph (KDD'03)
- Constraint-based mining of frequent patterns: Convertible constraints (ICDE'01), gPrune (PAKDD'03)
- Computing iceberg data cubes with complex measures: H-tree, H-cubing, and Star-cubing (SIGMOD'01, VLDB'03)
- Pattern-growth-based Clustering: MaPle(Pei, et al., ICDM'03)
- Pattern-Growth-Based Classification: Mining frequent and discriminative patterns (Cheng, et al, ICDE'07)

6.6. Đo lường tính lý thú của luật kết hợp - Đánh giá Mẫu

- Sau khi tìm được hàng ngàn mẫu, làm sao biết mẫu nào thực sự thú vị và có ý nghĩa?
 - Sinh ra quá nhiều luật, có nhiều luật không hay hoặc bị thừa
 - Cần đo lường tính lý thú để hạn chế luật

Trong 5,000 sinh viên:

- 3,000 chơi bóng rổ
- 3,750 ăn ngũ cốc
- 2,000 chơi bóng rổ và ăn ngũ cốc

	Basketball	Basketball	Sum
Cereal	2,000	1,750	3,750
Cereal	1,000	250	1,250
Sum	3,000	2,000	5,000

Chơi bóng rổ → Ăn ngũ cốc (40%, 66.7%): là sai lầm

Ví tỷ lệ sv ăn ngũ cốc là 75% > 66.7%

Chơi bóng rổ → Không ăn ngũ cốc (20%, 33.3%): có ý nghĩa thực tiễn hơn dù support và confidence thấp hơn

- Đây là một ví dụ kinh điển cho thấy confidence có thể gây hiểu lầm. Nó không tính đến tần suất xuất hiện của vế phải (kết luận) một cách độc lập.
- Chúng ta cần các thước đo có tính đến **tương quan thống kê (statistical correlation)**.
- Thước đo khách quan (Objective measures):** Dựa trên các tính toán thống kê từ dữ liệu.
 - Ví dụ: Support, confidence, correlation (tương quan), ...
- Thước đo chủ quan (Subjective measures):** Phụ thuộc vào người dùng và kiến thức miền của họ.
 - Người dùng khác nhau có thể thấy các mẫu khác nhau là thú vị.
 - Luật kết hợp lý thú nếu là điều mới lạ, gây ngạc nhiên, Có khả năng ứng dụng
 - Query-based (Dựa trên truy vấn):** Chỉ những mẫu liên quan đến yêu cầu của người dùng.
 - Judge against one's knowledge-base (So sánh với kiến thức nền):**
 - Unexpected (Bất ngờ):** Các mẫu đi ngược lại với những gì người dùng mong đợi.
 - Freshness, timeliness (Mới mẻ, kịp thời):** Các mẫu mới xuất hiện.

6.6.1. Lift (Độ nâng)

- Đo lường mức độ phụ thuộc/tương quan giữa hai sự kiện.

$$lift(B, C) = \frac{c(B \rightarrow C)}{s(C)} = \frac{s(B \cup C)}{s(B) \times s(C)}$$

- Lift(B, C) may tell how B and C are correlated
 - = 1: B and C are independent
 - > 1: positively correlated
 - < 1: negatively correlated

- **Áp dụng:**

	B	¬B	Σ _{row}
C	400	350	750
¬C	200	50	250
Σ _{col.}	600	400	1000

- $lift(B, C) = \frac{400 / 1000}{600 / 1000 \times 750 / 1000} = 0.89 \rightarrow B \text{ and } C \text{ are negatively correlated since } lift(B, C) < 1.$

- $lift(B, \neg C) = \frac{200 / 1000}{600 / 1000 \times 250 / 1000} = 1.33 \rightarrow B \text{ and } \neg C \text{ are positively correlated since } lift(B, \neg C) > 1.$

- Lift đối xứng: **Lift(B, C) = Lift(C, B).**

6.6.2. χ^2 (Chi-squared)

- Đây là một kiểm định thống kê cổ điển để kiểm tra sự độc lập giữa hai biến.

$$\chi^2 = \sum \frac{(Observed - Expected)^2}{Expected}$$

- **Observed:** Tần suất quan sát thực tế trong bảng chéo (contingency table).
- **Expected:** Tần suất kỳ vọng nếu hai biến là độc lập.

$$Expected(row, col) = (sum(row) * sum(col)) / total.$$

	B	¬B	Σ _{row}
C	400 (450)	350 (300)	750
¬C	200 (150)	50 (100)	250
Σ _{col}	600	400	1000

	B	¬B	Σ _{row}
C	400 (450)	350 (300)	750
¬C	200 (150)	50 (100)	250
Σ _{col}	600	400	1000

Expected value

Observed value

$$\chi^2 = \frac{(400 - 450)^2}{450} + \frac{(350 - 300)^2}{300} + \frac{(200 - 150)^2}{150} + \frac{(50 - 100)^2}{100} = 55.56$$

- Lookup χ^2 distribution table $\rightarrow B, C$ are correlated
- χ^2 -test shows B and C are negatively correlated since the expected value is 450 but the observed is only 400
- Thus, χ^2 is also more telling than the support-confidence framework

6.6.3. Lift and χ^2 : Are They Always Good Measures? Và bàn về các độ đo bất biến với rỗng

- **Null transactions (Giao dịch rỗng):** Các giao dịch không chứa cả B và C (tức là chứa $\neg B$ và $\neg C$).

	B	$\neg B$	Σ_{row}
C	100	1000	1100
$\neg C$	1000	100000	101000
$\Sigma_{col.}$	1100	101000	102100


null transactions

- $Lift(B, C) = 8.44$, một con số rất cao, cho thấy tương quan dương mạnh.
- $\chi^2 = 670$, cũng rất cao.
- **Mâu thuẫn:** Trực giác cho thấy B và C không có khả năng xảy ra cùng nhau, nhưng các thước đo lại chỉ ra điều ngược lại → *Too many null transactions may "spoil the soup"!*
- Điều này cho thấy chúng ta cần những thước đo **bất biến với giao dịch rỗng (null-invariant)**.
- *Null invariance means: The number of null transactions does not matter. Does not change the measure value (Một thước đo được gọi là bất biến rỗng nếu giá trị của nó không thay đổi khi số lượng giao dịch rỗng ($\neg A \neg B$) thay đổi).*
- **Tầm quan trọng của tính bất biến rỗng.**
 - Tại sao nó quan trọng? Vì trong dữ liệu giao dịch lớn, hầu hết các giao dịch có thể không chứa cả "sữa" và "cà phê".

	<i>milk</i>	$\neg milk$	Σ_{row}
<i>coffee</i>	<i>mc</i>	$\neg mc$	<i>c</i>
$\neg coffee$	$m \neg c$	$\neg m \neg c$	$\neg c$
Σ_{col}	<i>m</i>	$\neg m$	Σ

Data set	<i>mc</i>	$\neg mc$	$m \neg c$	$\neg m \neg c$	χ^2	<i>Lift</i>
D_1	10,000	1,000	1,000	100,000	90557	9.26
D_2	10,000	1,000	1,000	100	0	1
D_3	100	1,000	1,000	100,000	670	8.44
D_4	1,000	1,000	1,000	100,000	24740	25.75
D_5	1,000	100	10,000	100,000	8173	9.18
D_6	1,000	10	100,000	100,000	965	1.97

- Kết quả cho thấy giá trị Lift và χ^2 thay đổi một cách "điên cuồng" khi số lượng giao dịch rỗng thay đổi, trong khi mối quan hệ thực sự giữa m và c không đổi.
- → sự thiếu ổn định của Lift và χ^2 khi có nhiều giao dịch rỗng.

Measure	Definition	Range	Null-Invariant?
$\chi^2(A, B)$	$\sum_{i,j} \frac{(e(a_i, b_j) - o(a_i, b_j))^2}{e(a_i, b_j)}$	$[0, \infty]$	No
$Lift(A, B)$	$\frac{s(A \cup B)}{s(A) \times s(B)}$	$[0, \infty]$	No
$Allconf(A, B)$	$\frac{s(A \cup B)}{\max\{s(A), s(B)\}}$	$[0, 1]$	Yes
$Jaccard(A, B)$	$\frac{s(A \cup B)}{s(A) + s(B) - s(A \cup B)}$	$[0, 1]$	Yes
$Cosine(A, B)$	$\frac{s(A \cup B)}{\sqrt{s(A) \times s(B)}}$	$[0, 1]$	Yes
$Kulczynski(A, B)$	$\frac{1}{2} \left(\frac{s(A \cup B)}{s(A)} + \frac{s(A \cup B)}{s(B)} \right)$	$[0, 1]$	Yes
$MaxConf(A, B)$	$\max\left\{ \frac{s(A \cup B)}{s(A)}, \frac{s(A \cup B)}{s(B)} \right\}$	$[0, 1]$	Yes

- **Jaccard:** Đo lường mức độ chồng lấp giữa hai tập hợp. Rất phổ biến trong các bài toán so sánh độ tương tự.
- **Cosine:** Đo góc giữa hai vector.
- **Kulczynski:** Là trung bình cộng của hai độ tin cậy.
- **So sánh các thước đo bất biến rỗng với nhau:** Trở lại ví dụ sữa và cà phê:

All 5 are null-invariant

Data set	mc	$\neg mc$	$m \neg c$	$\neg m \neg c$	$AllConf$	Jaccard	Cosine	Kulc	MaxConf
D_1	10,000	1,000	1,000	100,000	0.91	0.83	0.91	0.91	0.91
D_2	10,000	1,000	1,000	100	0.91	0.83	0.91	0.91	0.91
D_3	100	1,000	1,000	100,000	0.09	0.05	0.09	0.09	0.09
D_4	1,000	1,000	1,000	100,000	0.5	0.33	0.5	0.5	0.5
D_5	1,000	100	10,000	100,000	0.09	0.09	0.29	0.5	0.91
D_6	1,000	10	100,000	100,000	0.01	0.01	0.10	0.5	0.99

Subtle: They disagree on those cases

- Trong các trường hợp D_1, D_2, D_3 , tất cả 5 thước đo đều cho kết quả khá giống nhau (đồng ý với nhau).
- Trong các trường hợp D_4, D_5, D_6 , chúng bắt đầu cho các kết quả khác nhau.
- Đặc biệt, thước đo **Kulc (Kulczynski)** giữ giá trị ổn định (0.5) trong cả ba trường hợp D_4, D_5, D_6 , trong khi các thước đo khác thay đổi. Giá trị 0.5 của Kulc thường biểu thị một mối quan hệ "trung tính" hoặc "không thiên vị".

6.6.4. Imbalance Ratio with Kulczynski Measure

- **IR (Imbalance Ratio - Tỷ lệ mất cân bằng)** = $\frac{|s(A) - s(B)|}{s(A) + s(B) - s(A \cup B)}$
- IR đo lường sự mất cân bằng về tần suất giữa hai itemset A và B.
- **Kết hợp Kulczynski và IR:**
 - **Kulczynski** cho biết mức độ tương quan trung bình.
 - **IR** cho biết mức độ cân bằng của mối quan hệ.
- Cặp thước đo (Kulc, IR) cung cấp một bức tranh toàn diện hơn về mối quan hệ giữa hai itemset.
- Ví dụ: một mối quan hệ có thể có tương quan trung bình, nhưng lại rất mất cân bằng (ví dụ: $A \rightarrow B$ rất mạnh, nhưng $B \rightarrow A$ rất yếu). Thông tin này rất hữu ích trong thực tế.

Data set	mc	$\neg mc$	$m\neg c$	$\neg m\neg c$	Jaccard	Cosine	Kulc	IR
D_1	10,000	1,000	1,000	100,000	0.83	0.91	0.91	0
D_2	10,000	1,000	1,000	100	0.83	0.91	0.91	0
D_3	100	1,000	1,000	100,000	0.05	0.09	0.09	0
D_4	1,000	1,000	1,000	100,000	0.33	0.5	0.5	0
D_5	1,000	100	10,000	100,000	0.09	0.29	0.5	0.89
D_6	1,000	10	100,000	100,000	0.01	0.10	0.5	0.99

- Phân tích D_4 , D_5 , D_6 :
 - D_4 : Kulc=0.5 (trung tính), IR=0 (cân bằng).
 - D_5 : Kulc=0.5 (trung tính), IR=0.89 (mất cân bằng).
 - D_6 : Kulc=0.5 (trung tính), IR=0.99 (rất mất cân bằng).

6.6.5. What Measures to Choose for Effective Pattern Evaluation?

- Trong các CSDL lớn, trường hợp giao dịch rỗng là rất phổ biến.
- Do đó, **tính bất biến rỗng (Null-invariance)** là một thuộc tính quan trọng.
- Khuyến nghị:**
 - Nếu giao dịch rỗng không chiếm ưu thế, Lift, χ^2 , và cosine là các thước đo tốt.
 - Ngược lại (trường hợp phổ biến hơn), nên sử dụng cặp **Kulczynski + Imbalance Ratio** để đánh giá độ thú vị của một mẫu.
- Việc lựa chọn thước đo không có câu trả lời "đúng cho mọi trường hợp", mà phụ thuộc vào đặc điểm của dữ liệu và câu hỏi nghiên cứu.

6.6.6. Một số Độ đo tính lý thú khác

- Interest:**
$$Interest = \frac{P(X,Y)}{P(X)P(Y)}$$
- Piatetsky-Shapiro's:**
$$PS = P(X,Y) - P(X)P(Y)$$
- ϕ - coefficient:**
$$\phi - coefficient = \frac{P(X,Y) - P(X)P(Y)}{\sqrt{P(X)[1 - P(X)]P(Y)[1 - P(Y)]}}$$

VD8: Độ đo tương quan Interest

- Interest < 1: X, Y tương quan nghịch
- Ngược lại, tương quan thuận

	Basketball	$\overline{\text{Basketball}}$	Sum
Cereal	2,000	1,750	3,750
$\overline{\text{Cereal}}$	1,000	250	1,250
Sum	3,000	2,000	5,000

Chơi bóng rổ → Ăn ngũ cốc

$$Interest(B, C) = \frac{P(B, C)}{P(B)P(C)} = \frac{\frac{2,000}{5,000}}{\frac{3,000}{5,000} * \frac{3,750}{5,000}} = 0.89$$

B: Basketball; C: Cereal

Chơi bóng rổ → Không Ăn ngũ cốc

$$Interest(B, \neg C) = \frac{P(B, \neg C)}{P(B)P(\neg C)} = \frac{\frac{1,000}{5,000}}{\frac{3,000}{5,000} * \frac{1,250}{5,000}} = 1.33$$

VD9: Độ đo tương quan PS

- Tea → Coffee:

$$\begin{aligned} & PS(Tea, Coffee) \\ &= P(Tea, Coffee) - P(Tea)P(Coffee) \\ &= 0.15 - 0.2 * 0.9 = -0.03 \end{aligned}$$

- ¬Tea → Coffee:

$$\begin{aligned} & PS(\neg Tea, Coffee) \\ &= P(\neg Tea, Coffee) - P(\neg Tea)P(Coffee) \\ &= 0.75 - 0.8 * 0.9 = 0.03 \end{aligned}$$

	Coffee	$\overline{Coffee}$	Sum
Tea	15	5	20
$\overline{Tea}$	75	5	80
Sum	90	10	100

symbol	measure	range	formula
ϕ	ϕ -coefficient	-1 ... 1	$\frac{P(A,B) - P(A)P(B)}{\sqrt{P(A)P(B)(1-P(A))(1-P(B))}}$
Q	Yule's Q	-1 ... 1	$\frac{P(A,B)P(\overline{A},\overline{B}) - P(A,\overline{B})P(\overline{A},B)}{P(A,B)P(\overline{A},\overline{B}) + P(A,\overline{B})P(\overline{A},B)}$
Y	Yule's Y	-1 ... 1	$\frac{\sqrt{P(A,B)P(\overline{A},\overline{B})} - \sqrt{P(A,\overline{B})P(\overline{A},B)}}{\sqrt{P(A,B)P(\overline{A},\overline{B})} + \sqrt{P(A,\overline{B})P(\overline{A},B)}}$
k	Cohen's	-1 ... 1	$\frac{P(A,B) + P(\overline{A},\overline{B}) - P(A)P(B) - P(\overline{A})P(\overline{B})}{1 - P(A)P(B) - P(\overline{A})P(\overline{B})}$
PS	Piatetsky-Shapiro's	-0.25 ... 0.25	$P(A,B) - P(A)P(B)$
F	Certainty factor	-1 ... 1	$\max(\frac{P(B A) - P(B)}{1 - P(B)}, \frac{P(A B) - P(A)}{1 - P(A)})$
AV	added value	-0.5 ... 1	$\max(P(B A) - P(B), P(A B) - P(A))$
K	Klogsen's Q	-0.33 ... 0.38	$\sqrt{P(A,B)} \max(P(B A) - P(B), P(A B) - P(A))$
g	Goodman-kruskal's	0 ... 1	$\frac{\sum_j \max_k P(A_j, B_k) + \sum_k \max_j P(A_j, B_k) - \max_j P(A_j) - \max_k P(B_k)}{2 - \max_j P(A_j) - \max_k P(B_k)}$
M	Mutual Information	0 ... 1	$\frac{\sum_i \sum_j P(A_i, B_j) \log \frac{P(A_i, B_j)}{P(A_i)P(B_j)}}{\min(-\sum_i P(A_i) \log P(A_i) \log P(A_i), -\sum_i P(B_i) \log P(B_i) \log P(B_i))}$
J	J-Measure	0 ... 1	$\max(P(A, B) \log(\frac{P(B A)}{P(B)}) + P(\overline{A}\overline{B}) \log(\frac{P(\overline{B} \overline{A})}{P(\overline{B})}), P(A, B) \log(\frac{P(A B)}{P(A)}) + P(\overline{A}\overline{B}) \log(\frac{P(\overline{A} \overline{B})}{P(\overline{A})})$
G	Gini index	0 ... 1	$\max(P(A)[P(B A)^2 + P(\overline{B} \overline{A})^2] + P(\overline{A})[P(B \overline{A})^2 + P(\overline{B} \overline{A})^2] - P(B)^2 - P(\overline{B})^2, P(B)[P(A B)^2 + P(\overline{A} B)^2] + P(\overline{B})[P(A \overline{B})^2 + P(\overline{A} \overline{B})^2] - P(A)^2 - P(\overline{A})^2)$
s	support	0 ... 1	$P(A, B)$
c	confidence	0 ... 1	$\max(P(B A), P(A B))$
L	Laplace	0 ... 1	$\max(\frac{NP(A,B)+1}{NP(A)+2}, \frac{NP(A,B)+1}{NP(B)+2})$
IS	Cosine	0 ... 1	$\frac{P(A,B)}{\sqrt{P(A)P(B)}}$
γ	coherence(Jaccard)	0 ... 1	$\frac{P(A,B)}{P(A)+P(B)-P(A,B)}$
α	all_confidence	0 ... 1	$\frac{\max(P(A), P(B))}{P(A,B)}$
o	odds ratio	0 ... ∞	$\frac{P(A,B)P(\overline{A},\overline{B})}{P(\overline{A},B)P(A,\overline{B})}$
V	Conviction	0.5 ... ∞	$\max(\frac{P(A)P(\overline{B})}{P(\overline{A}\overline{B})}, \frac{P(B)P(\overline{A})}{P(\overline{B}\overline{A})})$
λ	lift	0 ... ∞	$\frac{P(A,B)}{P(A)P(B)}$
S	Collective strength	0 ... ∞	$\frac{P(A,B)+P(\overline{A}\overline{B})}{P(A)P(B)+P(\overline{A})P(\overline{B})} \times \frac{1-P(A)P(B)-P(\overline{A})P(\overline{B})}{1-P(A,B)-P(\overline{A}\overline{B})}$
χ^2	χ^2	0 ... ∞	$\sum_i \frac{(P(A_i) - E_i)^2}{E_i}$